EM1 — Culture Fit Q&A

Formal round with Gunnar (CTO) | Behavioral + Values + Motivation

> **How to use this doc:** These are the questions behind the questions. Every culture fit answer should signal three things: ownership mindset, comfort with ambiguity at small scale, and genuine alignment with the mission. Keep answers to 60-90 seconds. Concrete, specific, no platitudes.

Q: Why EM1? Why now?

The honest answer — three reasons:

1. The mission is specific and real. Most SaaS companies say they're making the world better. EM1 is helping emergency management agencies write better plans that determine how a community responds to a disaster. That's not abstract impact — a FEMA-compliant HMP that gets approved is an agency that has its preparedness funding. I want to build things where the outcome is that legible.

2. The technical fit is exact, not approximate. The TrustX pipeline I built at Samay — evidence collection, LLM scoring, human-in-the-loop review with citations — is structurally the same problem as EM1's plan evaluation. I know what breaks in production on that class of system. That's not a "transferable skill" claim; it's the actual thing. I'd contribute from week 2, not month 4.

3. The stage. 4 engineers, real customers, architectural decisions being made right now that will define the system for 5 years. This is the moment I want to be in the room for. I've been the sole engineer at that stage and I know how to move fast and build things that hold up. EM1 is the same stage with more domain complexity and a team — which is better.

Q: Tell me about yourself. Walk me through your background.

Opening (30 seconds, then let them steer):

I'm a full-stack engineer with a focus on production LLM systems. Most recently I was the founding engineer at Samay Systems — I joined as their first technical hire, designed the full architecture, and built TrustX, a compliance automation platform, from zero to three paying B2B customers over 9 months. That meant everything: CDK infrastructure across 15+ AWS stacks, the evidence-collection pipeline across 6 data sources, LLM scoring with hallucination prevention, multi-tenant isolation, and the React dashboard. Before that I led the AI/ML engineering team at Epik Solutions — 6 engineers, LangGraph agentic workflows in production, evaluation infrastructure with Langfuse. And before that Cardinal Health, where I built production data pipelines on PostgreSQL, AWS, and Terraform at enterprise scale.

The thread across all of it: I gravitate toward the AI-in-production problem — not prototypes, but systems where wrong LLM outputs have real consequences and you've built the infrastructure to catch them.

Q: What motivates you day-to-day as an engineer?

The real answer:

Two things.

First, closing the loop. I get genuine satisfaction from seeing a feature go from a rough requirement to a deployed thing a user is interacting with — and from monitoring it long enough to know it worked. At Samay, the "Test Control" button took 5 days start to finish: React state, Flask API, Lambda worker, CDK infra. Watching a compliance team click it and see a grounded LLM result in real time — that's the loop I want to close.

Second, building things that are honest. The evidence-gating pattern I built at Samay — if the LLM can't find verbatim evidence, it returns INCONCLUSIVE instead of a confident wrong answer — came directly from caring about the downstream use. An auditor defending a report in front of a regulator needs to be able to trust what the system says. That constraint made the system harder to build and more valuable to use. I want to work on problems where that distinction matters.

Q: How do you work on a small team? What's different from a larger org?

What I've learned from both contexts:

At Samay, I was the only engineer — the context was total but so was the accountability. Every architectural decision was mine. The lack of review pressure means you have to build your own review discipline: decision logs, self-PR review with written notes, handoff-quality documentation throughout. You can't afford a 6-week architecture review cycle, but you also can't afford a decision you can't explain 6 months later.

At Epik, 6 engineers. The main skill is parallelism — how do you decompose work so 6 people can move without blocking each other? Short-lived feature branches, tight PR scope (< 400 lines), clear ownership of modules so two people aren't touching the same area.

For EM1 at 4 engineers: I'd expect a lot of the Samay discipline — move fast, own it fully, write things down for the team — with the Epik benefit of having someone to catch your blind spots. That's actually the best of both. I don't need a lot of process, but I do write things down.

Q: How do you handle working with non-technical domain experts or stakeholders?

Story (Samay compliance auditor):

At Samay, the founders brought in an experienced compliance auditor to review our LLM evaluation outputs. He'd conducted hundreds of SOC2 audits. My first instinct was to explain the system — how the pipeline worked, what the evidence-gating did. His first reaction to our outputs was: "This is technically correct, but I couldn't defend this in front of a regulator. The quote is too vague. I need a clause reference."

That single comment rewrote my mental model. I'd been optimizing for technical correctness. He was optimizing for something harder: a written artifact that would hold up to scrutiny from a skeptical third party. Those aren't the same problem.

I spent the next three hours asking him to talk through 20 outputs and explain what would make each one defensible or not. I took notes on the vocabulary he used: "maps to a named sub-requirement," "shows pattern of behavior," "source dated within the audit period." Then I rewrote the prompt architecture around those criteria. No amount of reading the SOC2 standard would have given me that — you learn expert criteria by watching an expert work.

For EM1, the same principle applies: the emergency management practitioners on the team know what a useful plan evaluation looks like better than I do. My job in the first month is to understand that, not to assume I already do.

Q: How do you handle ambiguity — when requirements aren't fully defined?

What I actually do:

I start by separating "ambiguous requirements" from "genuinely unknown." Most ambiguity at small companies isn't intentional — it's that the founders haven't needed to write it down yet because they've been carrying it in their heads. The fix is usually one focused conversation and a written summary back: "Here's what I heard. Here's what I'm building. Tell me what I got wrong."

At Samay, I had near-total autonomy on architecture. The requirement was roughly "automate SOC2 evidence collection and evaluation." That's ambiguous. My approach: build the smallest thing that touches the real problem, show it to the domain expert (the compliance auditor), and let the feedback sharpen the requirements. The first version was one hardcoded Lambda, one control, one data source, rough React output. But it was real — it ran against real Jira data and produced a verdict. Real trumps specification.

What I don't do: wait for clarity before starting or build the fully abstract version upfront. Neither serves a small team. Start concrete, make the ambiguity visible fast, iterate.

Q: Tell me about a time you disagreed with a direction. How did you handle it?

Story (Workato vs. Lambda, Epik):

At Epik, a client's IT team wanted to use Workato — a no-code integration platform — to orchestrate our entire LLM workflow. Their reasoning: their team could maintain it without needing engineers on every change.

I thought it was the wrong tool. But I didn't say that in the meeting. I spent a weekend building a proof of concept of our phishing triage workflow in Workato — specifically to understand where the blockers were and what they cost us. Specific findings: Workato recipes are sequential (our workflow needed parallel lookups with a merge step, not possible cleanly), prompt templates lived

in a UI field with no version control or PR review, and Langfuse tracing was completely absent — we'd lose observability on every LLM call.

Brought those findings to the client IT lead and my manager. Not "this is wrong" — "here's what works and what doesn't, and here's the cost of what doesn't." Proposed a split: Workato for upstream data collection where their team had genuine ownership, LangGraph for the AI evaluation step where we needed control. They agreed. The ownership boundary was clean.

The principle: push back with evidence, not instinct. And offer a path forward, not just a rejection.

Q: How do you prioritize when everything feels urgent?

My framework:

Two-axis: operational risk vs. velocity impact.

Things that cause production incidents or could lose customer data — those are always now. Everything else gets categorized by: "what's blocked on this?" If three engineers can't move forward without this being resolved, that's next. If the impact is only future work, it goes on the backlog with a clear note.

At Samay, this showed up in the technical debt audit. Some resources were console-managed and not in CDK — unrecoverable if lost (operational risk: now), some had manual modifications that drifted from CDK state (operational risk: schedule), some were aesthetic (no risk: background). I paid debt in that order and never interrupted product work for the aesthetic category.

For a small team like EM1: I'd communicate priority decisions explicitly. Not "I'm doing X instead of Y" without context — "I'm doing X first because Y unblocks Gunnar's feature" or "this is the thing that would be painful to debug at 2am." Make the reasoning visible.

Q: Remote work — how do you stay effective and connected working fully async?

What works for me:

I've worked fully remote in both Samay and Epik contexts. The async discipline matters more than the tooling.

The things I actually do:

- **Over-document decisions in writing.** In-person, you can grab someone in the hallway and explain a tricky tradeoff. Remote, that conversation has to live in a PR comment, a ticket, or a Slack thread — or it evaporates. I write PR self-reviews that explain why I made the choices I did, not just what I did.
- **Time-box ambiguity.** If I'm stuck for more than 30-40 minutes and I think a 5-minute conversation would unstick it, I ask. I don't grind for 4 hours out of reluctance to interrupt.
- **Update the team proactively.** When I finish a significant chunk, I send a brief written summary — not a status update theater, but "here's what I shipped, here's what changed,

here's what I'm doing next." Keeps context shared without requiring synchronous meetings.

- **Own my scope completely.** Remote work breaks down when people are waiting for each other to clarify something that could have been documented upfront. I try to eliminate those blockers at the start of a task: write the interface first, agree on the contract, then both parties can work.

The twice-yearly offsite model EM1 uses is one I've seen work well — async for execution, synchronous for alignment on direction and relationship building. That rhythm makes sense to me.

Q: What kind of engineering culture do you want to work in?

What I actually value:

- **Ownership with trust.** I do my best work when I own a problem fully — not just the implementation but the design decision and the production monitoring. That requires trust from the team that I'll flag when I'm uncertain and document when I'm confident.
- **Small PRs, real code review.** Code review is how a team shares a mental model of the codebase. It's not a gate — it's a conversation. I want reviews that ask "why this approach" not just "does it compile." And I want to give the same.
- **Writing things down.** The Samay handoff document — the founders used it in their hiring process as a selling point for candidates. That's what written engineering culture buys you: institutional knowledge that doesn't evaporate when someone leaves.
- **Mission first, process second.** I've worked in orgs with too much process and too little. The right amount is: enough structure that nothing falls through the cracks, minimal enough that an engineer doesn't spend more time in Jira than in the editor. For a 4-person team, that's probably lightweight tickets, async standups, and one weekly sync.

Q: What are you looking to grow in over the next 1-2 years?

Honest answer:

Two areas.

Depth in the emergency management domain. I can build the LLM systems and infrastructure — those are strengths I'm confident in. The part I'd be learning is the actual domain: how emergency managers use plans, what makes an HMP evaluation credible to a FEMA reviewer, where practitioners feel the current tools fall short. That learning happens by sitting close to the EM1 operator team and treating every piece of domain feedback like the compliance auditor conversation at Samay.

Going further up the product architecture curve. At Samay I made all the decisions — but in isolation. At EM1 with a small team and real customers, I'd want to develop the skill of making those decisions in a collaborative context: understanding Gunnar's product vision, contributing to roadmap conversations, not just executing against a spec. That's the difference between a strong IC and an engineer who makes the team's trajectory better.

Q: What does success look like for you in the first 90 days here?

My answer:

Three things:

Month 1: Understand the codebase well enough that I can make a meaningful contribution without breaking existing behavior. Know the data model, the LLM pipeline, the deployment setup. Ship something small — a bug fix, a small feature — that passes code review on first submission. That's the calibration exercise.

Month 2: Own a feature end-to-end. Design it, build it, deploy it, monitor it. Write the PR description that explains why the tradeoffs were made the way they were. Ask the hard questions about edge cases before they hit production.

Month 3: Start contributing to the architectural direction — not just "here's my implementation" but "here's how I'd extend this to handle the next use case." Be the person on the team who asks "what happens when we go from 60 agencies to 600?" before it's a crisis.

The underlying measure: by 90 days, Gunnar shouldn't need to hold my context for me. I should be generating context and sharing it.

Q: What do you do when you're in over your head technically?

Honest answer:

It happens. The question is what you do with it.

My pattern: I time-box the stuck window. If I've been unclear on something for more than a day, I write down exactly what I know and what I don't — the act of writing usually surfaces the gap. Then I either find the answer (read the source, run an experiment, find the right documentation) or I ask, with the specific question prepared, not the vague "I'm stuck."

Examples: Neo4j Cypher was new to me at Samay. I had two weeks of performance data saying I needed it; I didn't have the skill. I built a small sandbox graph, implemented the identity traversal in Cypher with a reference query open, benchmarked against my PostgreSQL CTE, and then deployed. The learning was targeted — not "learn Neo4j broadly" but "learn enough Cypher to implement and benchmark this specific traversal pattern."

Terraform is the current example at EM1. I've used CDK extensively, and I understand the underlying resource graph well — the mental model transfers. I'd ramp on the HCL syntax and EM1-specific module patterns in the first weeks by reading the existing infra code, not from docs first.

Q: How do you approach code review — giving and receiving?

Giving review:

I review for two things: correctness and comprehensibility. Correctness: does this handle the edge cases? Does the logic actually do what the PR description says? Comprehensibility: will I understand this in 6 months without the author here to explain it? If the answer to either is no, that's a comment.

I write comments as questions when I'm uncertain ("is there a reason we're not using X here?") and as suggestions with reasoning when I have a clear preference ("I'd use a partial index here instead — it'll only scan the PENDING rows and should be ~40x smaller"). I don't nitpick style in code review — that's what a linter is for.

Receiving review:

My strong prior is: the reviewer is probably right that something is unclear, even if their specific suggestion isn't the right fix. If someone is confused by my code, that's information — the code was confusing. I don't defend my first implementation. I fix the confusion and explain the tradeoff in the PR if the decision was deliberate.

The one thing I push back on: when a review comment is about aesthetics or personal style preference with no functional implication. I'll explain my reasoning, but I don't rewrite working code to match a style the linter doesn't enforce.

Q: EM1 works with emergency management practitioners, not just software engineers. How do you feel about that?

Why I'm drawn to it:

The compliance auditor experience at Samay was the best technical feedback I ever got — and it came from someone who knew nothing about Python or vector databases. He knew compliance, and what he told me about "defensible evidence" made the system better in ways that no code review would have caught. That's the dynamic I'm drawn to: practitioners who understand the domain deeply, engineers who understand the implementation. The best products come from the intersection.

For EM1, the emergency management operators on the team are the equivalent of that auditor. They know what a credible plan looks like, what an evaluator will actually scrutinize, where practitioners currently waste the most time. My job is to build systems that serve their expertise, not substitute for it. That's also why I believe in human-in-the-loop architectures — the LLM handles the scale problem, the practitioner handles the judgment problem. Neither replaces the other.

Q: What questions do you have for me?

Pick 3 — listed in priority order:

1. **The operator + engineer dynamic:** How does the collaboration work in practice between

the EM1 operators and the engineering team? Are they involved in feature scoping, testing, ongoing feedback? I want to understand how domain expertise flows into the product decisions.

2. **Biggest technical challenge right now:** What's the hardest unsolved problem on the engineering side today — not the feature on the roadmap, but the thing that keeps you up? That tells me where the real work is.
3. **How the team makes architectural decisions:** With 4 engineers, how do big calls get made — is it a conversation among the team, does Gunnar drive the direction, do you use RFCs or ADRs? I want to understand the collaboration structure before I'm in it.
4. *(If time)* **What does success look like for this hire in 6 months from your perspective?** I want to calibrate to your measure, not just mine.